

学 号:	1023006395 1023000356 1023000344 1023000175
------	--

武汉理工大学

软件工程实践（二）

学 院	计算机与人工智能学院
专 业	软件工程
班 级	软件 2302
姓 名	刘紫媛，殷瑞涵，赵欣然，谢天
编 号	
指导教师	唐祖锴

2026 年 6 月 19 日

目录

1. 任务概述	1
2. 小组任务计划.....	1
2.1 任务分析.....	2
2.2 项目架构与功能设计.....	3
2.2.1 系统总体架构	3
2.2.2 核心功能模块	4
2.3 团队组成与分工.....	6
2.4 软件工具与项目约束.....	7
2.4.1 开发工具与技术栈	7
2.4.2 项目开发流程约束	7
3. 小组软件过程.....	9
3.1 代码仓库.....	9
3.1.1 仓库基本信息	9
3.1.2 仓库目录结构	9
3.1.3 仓库文档与可追踪性	10
3.2 软件版本计划.....	11
3.2.1 版本规划原则	11
3.2.2 阶段性版本安排	11
3.2.3 里程碑管理	11
3.3 开发分支模型与提交流程.....	12
3.3.1 分支模型设计	12
3.3.2 提交流程	13
3.3.3 提交说明规范	13
3.4 代码规范与检查流程.....	14
3.4.1 Java 代码规范	14
3.4.2 前端代码与资源规范	15
3.4.3 注释与文档规范	15
3.4.4 Pull Request 人工检查流程	15
3.4.5 自动化检查流程	16
3.5 项目测试计划与实施流程.....	16
3.5.1 测试目标	16
3.5.2 自动化测试范围	17
3.5.3 前端与接口测试	17
3.5.4 测试实施流程	18
3.5.5 缺陷修复闭环	18
3.6 项目集成与发布规范.....	18
3.6.1 集成原则	18
3.6.2 Maven 构建与打包	19
3.6.3 Docker 容器化发布	19

3.6.4	Render 前后端部署	19
3.6.5	发布前检查规范	20
3.6.6	发布成果	21
4.	小组成员任务报告	21
4.1	成员一（刘紫媛）任务分工及实施过程描述	21
4.1.1	工作内容概述	21
4.1.2	核心模型与游戏规则	21
4.1.3	Web 游戏框架与状态协同	21
4.1.4	可视化交互与重点功能	22
4.1.5	游戏功能扩展与交互体验优化	23
4.1.6	测试、持续集成与部署	23
4.1.7	工作小结	24
4.2	成员二（殷瑞涵）任务分工及实施过程描述	24
4.2.1	任务概述	24
4.2.2	实施过程描述	25
4.2.3	最终成果总结	27
4.3	成员三（赵欣然）任务分工及实施过程描述	27
4.3.1	任务概述	27
4.3.2	实施过程	27
4.3.3	实施成果	29
4.3.4	总结	30
4.4	成员四（谢天）任务分工及实施过程描述	30
5.	实施过程问题记录与分析	31
5.1	前后端状态同步冲突导致物品丢失	31
5.2	加速移动时房间交界处卡住	32
5.3	丢弃物品出现在当前房间而非初始房间	32
5.4	返回上一房间功能失效	32
5.5	后端传送房间规则的问题	33
5.6	后端物品属性扩展与 API 适配	33
5.7	自动存档机制不完善	33
6.	任务总结	34
7.	参考文献	36

1. 任务概述

本项目以经典教学案例 World of Zuul 为基础，围绕“校园探索与解谜冒险”主题，设计并开发了一套基于 Java Web 技术的可视化游戏系统。项目突破了传统 Zuul 游戏仅依赖命令行交互的模式，在保留其面向对象设计思想和核心游戏机制的基础上，引入浏览器图形界面、前后端交互、用户管理、游戏存档以及数据库持久化等功能，将原有文本冒险程序扩展为具有完整工程结构和实际可玩性的 Web 游戏项目。

游戏以校园环境为背景，构建了校园入口、主大厅、图书馆、计算机实验室、森林小路、上锁宝库、宝藏房间以及传送房间等多个场景。玩家需要不同房间之间进行探索，通过观察环境、收集物品、分析线索和破解谜题逐步推进游戏进程，最终获得关键道具并完成逃离任务。整个游戏流程围绕“探索—收集—推理—解锁—通关”的主线展开，既保留了 Zuul 项目原有的冒险特色，又增强了游戏的交互性和趣味性。

系统采用分层设计思想进行开发，主要由前端展示层、Web 服务层、游戏控制层、游戏逻辑层、状态管理层和数据持久化层组成。前端使用 HTML、CSS 和 JavaScript 实现房间场景展示、角色动画、地图查看、背包管理、物品交互以及谜题弹窗等功能；后端基于 Java 构建轻量级 HTTP 服务，通过 REST API 与前端进行通信；游戏逻辑层负责房间、玩家、物品和命令系统的实现；状态管理层负责游戏进度保存与恢复；数据库层则负责用户信息、游戏记录以及玩家状态的持久化存储。

项目开发过程中不仅实现了游戏核心功能，还重点关注软件工程实践，包括模块化设计、前后端分离架构、数据库设计、自动化测试以及代码规范管理等内容。通过本项目的开发与完善，最终形成了一套集游戏逻辑、用户交互、数据管理和系统测试于一体的完整校园冒险游戏平台，实现了从传统教学案例向实际软件项目的扩展与升级，同时也为团队成员综合运用面向对象程序设计、数据库技术、Web 开发技术以及软件工程方法提供了实践平台。

2. 小组任务计划

本章主要针对“软件工程实践二”协同开发项目的核心内容进行全面分析。通过对实践任务的深度解构，明确项目在技术架构、团队协同、分支管理、CI/CD 流水线以及软件工程质量保障等方面的具体实施方案，并制定严格的项目管理策略与强制约束，确保项目在有限的交付周期内高质量、高标准地完成。

2.1 任务分析

本次实践的核心任务，是在软件工程实践一的基础上，以小组协同开发的方式完成一个具有完整交互界面和功能逻辑的软件系统。实践重点涵盖软件架构设计、小组软件过程与项目管理、软件分支管理、软件代码评审以及持续集成与部署五个方面。小组需要基于 GitHub 平台开展分布式协同开发，掌握 Maven 项目管理机制，并了解 DevOps 软件开发流程。

本小组选取的开发项目为“World of Zuul: Campus Adventure”——一款基于经典文本冒险游戏 World of Zuul 扩展而来的校园探索解谜游戏。原版 World of Zuul 由 Will Crowther 于 20 世纪 70 年代开发，后经 Don Woods 扩充，是一款以洞穴探险为主题的经典文字冒险游戏。本小组在保留其面向对象结构和命令式交互核心的基础上，将其升级为一个具有 Web 可视化体验的全栈游戏应用。

本次实践的工作重点主要包括以下几个方面：

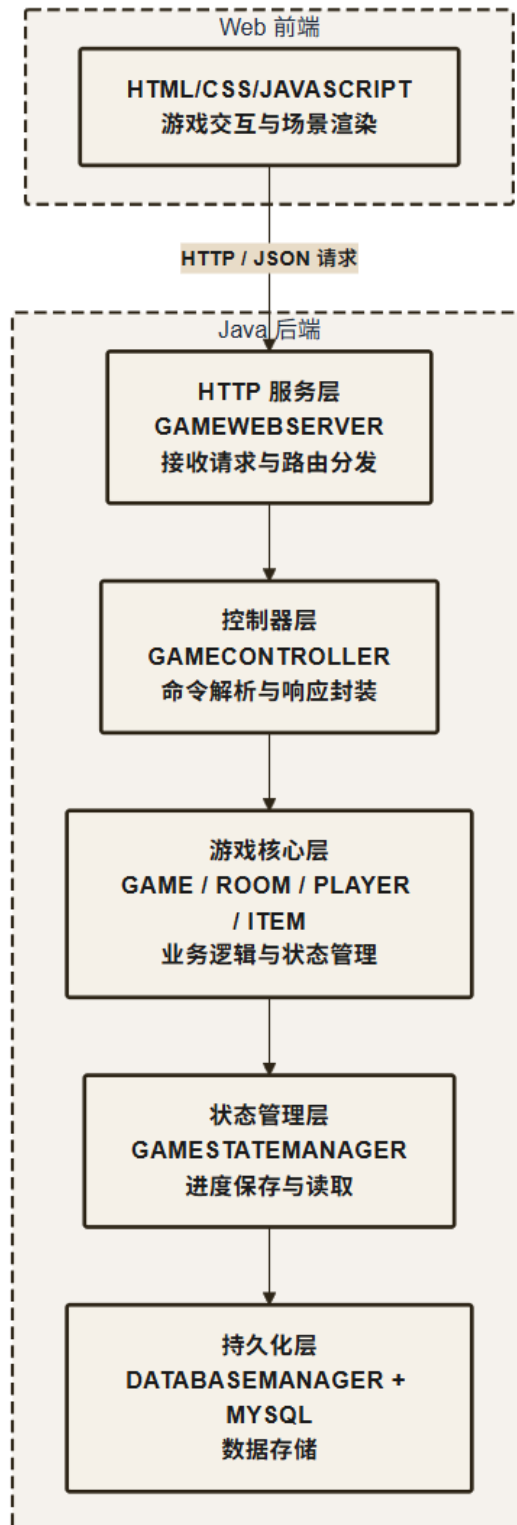
（1） 软件架构设计。项目需要从原有的单体命令行程序扩展为分层清晰的全栈 Web 应用，涵盖前端交互层、HTTP 服务层、控制器层、游戏核心层、状态管理层和持久化层。各层之间需要通过合理的接口设计实现解耦，确保系统的可维护性和可扩展性。

（2） 小组协同开发。小组四人需要基于 GitHub 平台开展分布式协同开发，遵循约定的分支模型进行功能开发、代码测试、提交、归并与同步等^{[6][7]}。每位成员需要在各自的工作分支上完成开发任务，并按照规定格式填写代码提交说明。

（3） 功能扩充与完善。样例工程仅具备最基本的程序功能框架，小组需要确定不少于 5 项结构优化或功能扩充需求。本项目的扩充方向包括：Web 可视化界面、角色行走动画、物品交互系统、谜题流程系统、用户会话与登录注册、游戏进度保存与读取、后端 API 服务等。

（4） 工程化实践。需要运用 Maven 进行项目工程管理，利用 GitHub Actions 实现自动化代码规范检查、自动化测试和自动化打包发布^[8]。

2.2 项目架构与功能设计



2.2.1 系统总体架构

项目采用分层架构设计，整体分为六个层次：

层次	职责	核心组件
Web 前端	场景渲染、角色动画、按钮控制、弹窗与玩家反馈	index.html、game.js、style.css
HTTP 服务层	接收浏览器请求，返回 JSON 数据	GameWebServer.java
控制器层	解释命令、调用游戏逻辑、组织响应数据	GameController.java
游戏核心层	房间、出口、玩家、物品、命令与通关状态管理	Game.java、Room.java、Player.java、Item.java、*Command.java
状态管理层	游戏进度的保存、读取与同步	GameStateManager.java
持久化层	用户与游戏状态的数据库存储支持	DatabaseManager.java + MySQL

前端与后端通过 HTTP API 进行通信，前端发起请求后由 GameWebServer 接收并路由至 GameController，控制器解析命令后调用游戏核心层的业务逻辑，将结果通过 JSON 格式返回前端进行渲染。

2.2.2 核心功能模块

(1) 游戏世界模块

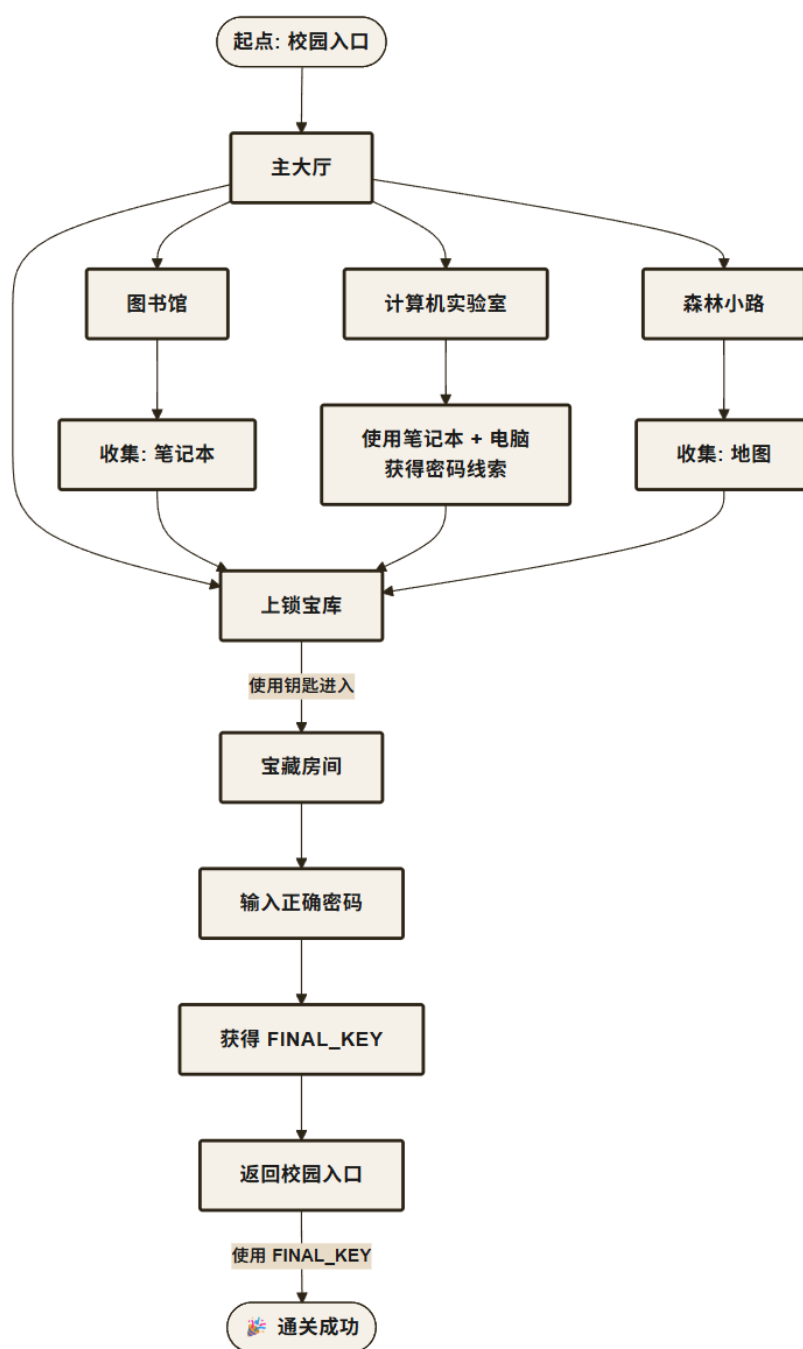
游戏以大学校园为舞台，设计了多个相互连接的探索场景：

- 校园入口：游戏起点，也是最终逃离位置
- 主大厅：校园核心枢纽，连接多个区域
- 图书馆：放置笔记本线索，推动谜题链
- 计算机实验室：电脑与数据线相关谜题区域
- 森林小路：地图、密码箱和最终出口相关区域
- 上锁宝库：需要钥匙解锁的重要房间
- 宝藏房间：后期关键奖励区域
- 传送房间：确认后随机传送到其他房间

(2) 物品与背包系统

物品具有名称、重量、描述、图标、类型和使用效果等属性。玩家可以在房间中拾取物品、丢弃物品、查看物品详情并使用物品。房间内物品以悬浮气泡形式直接显示，玩家可点击图标拾取。

(3) 谜题系统



游戏围绕“探索—收集—推理—解锁—通关”展开连续解谜流程：

- 从校园入口开始探索，熟悉房间连接关系
- 收集钥匙、地图、笔记本、数据线等关键物品
- 使用钥匙进入上锁宝库
- 通过笔记本和电脑获得谜题线索
- 输入正确密码打开箱子，获得 final_key

f) 回到最终出口，使用最终钥匙完成游戏

(4) 交互与控制模块

支持方向按钮、WASD 键和方向键三种操作方式。方向按钮仅在当前房间实际可走时启用，避免误导玩家。角色会根据移动方向切换对应的行走动画帧，提供自然的移动反馈。

(5) 用户与存档模块

提供登录、注册功能以及 session 级的游戏状态管理。支持游戏进度的保存、读取和新游戏状态重置，提供三槽本地存档功能。

(6) 后端 API 服务

通过 GameWebServer 和 GameController 为前端提供完整的 HTTP API 服务，支持游戏状态查询、命令执行、存档读写等操作。

2.3 团队组成与分工

本小组由四人组成，推选刘紫媛担任组长，负责组织、协调和领导团队开发。各成员及其对应的开发分支如下：

姓名	角色	开发分支	主要职责
刘紫媛	组长	feature/coreworldplayeritems	核心底层逻辑、前后端框架搭建、视觉资源接入、移动与前端交互、CI/CD 自动检查打包、部署配置
殷瑞涵	组员	feature/corecommandinteractions	后端命令系统框架设计与核心实现、谜题系统构建、系统联调与状态同步优化及 Bug 修复
赵欣然	组员	feature/uiqualitydevopsdocs	Web 前端与 Java GUI 开发、静态资源整理、交互功能完善以及 Bug 修复
谢天	组员	feature/servicepersistenceapi	后端 API 服务搭建、数据库持久化实现、存档读档功能开发及测试用例编写

2.4 软件工具与项目约束

2.4.1 开发工具与技术栈

本项目采用的主要技术与工具如下：

类别	技术/工具	用途
编程语言	Java 8+	后端业务逻辑开发
前端技术	HTML5 / CSS3 / JavaScript	Web 界面开发
构建工具	Apache Maven	项目工程管理与构建
版本控制	Git / GitHub	代码版本管理与协同开发
持续集成	GitHub Actions	自动化代码检查、测试与打包
数据库	MySQL / JDBC	用户与游戏状态持久化存储
部署平台	Render + TiDB Cloud	应用部署与云数据库服务
文档编制	MarkDown	README 及项目文档编写

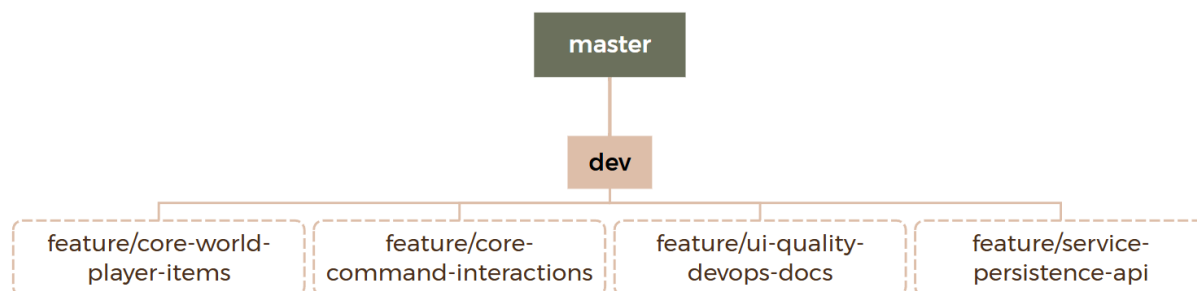
2.4.2 项目开发流程约束

为保障小组协同开发的质量与效率，确保项目在既定时间节点内高质量交付，项目遵循以下管理策略与强制约束规范。这些约束贯穿于整个开发生命周期，涵盖版本控制、代码质量、项目管理及交付物等多个维度。

（1）分支管理策略

小组采用基于功能分支的协同开发模型。每位成员在各自的功能分支（**feature/***）上进行开发，完成功能后通过 **Pull Request** 向开发分支（**dev**）合并。主分支（**master**）始终保持可部署状态。具体要求包括：

- 所有功能开发均在个人功能分支完成
- 工作分支在合并前须同步到远程仓库，供教师检查个人开发进度
- 代码提交须按照小组约定的规范格式填写提交说明



（2） 代码质量管理

代码质量是本项目关注的核心要素之一。小组制定了统一的 Java 编码规范，涵盖命名约定、代码格式、注释规范等方面，并利用 GitHub Actions 在代码提交时自动触发代码格式检查，对于不符合规范的提交系统将自动给予失败提示，从而在源头上保证代码风格的一致性。与此同时，小组要求所有源代码包含规范化注释和必要的说明文档，以提升代码的可读性与可维护性。在功能测试方面，项目基于 JUnit 编写了覆盖核心业务逻辑的单元测试与集成测试用例，同样通过 GitHub Actions 在每次提交时自动触发测试用例检查，未能通过测试的代码将被拒绝合并，确保新功能的引入不会破坏既有功能的正确性^[9]。自动化检查与人工评审相结合的质量保障体系，有效地降低了缺陷引入的风险。

（3） 项目管理约束

项目管理遵循敏捷开发的思想，将整体开发任务拆解为若干个可独立交付的功能模块，并为每个模块设定了明确的里程碑时间节点。所有开发任务均通过 GitHub 平台创建 Issue 进行跟踪管理，每位组员被分配明确的工作任务，任务的完成情况通过 Issue 状态和关联的 Pull Request 进行闭环管理。工作任务的划分不仅有助于明确个人职责边界，也是最终衡量小组成员工作量的重要依据。小组定期开展沟通讨论，同步各模块的进展情况，及时发现并协调解决开发过程中遇到的技术难题与协作问题，确保项目整体按计划有序推进。所有开发工作成果须在代码仓库中提交，个人的实际贡献将以代码仓库中体现的工作量为主要评判依据，包括提交次数、代码行数、Issue 参与度等多维度指标。

（4） 交付物约束

项目需在截止日期前完成以下交付：

- 小组代码仓库中的完整源代码与提交记录，要求仓库结构清晰、提交日志规范
- 修改完善的 README.md 项目介绍文档，参照开源社区标准编写，内容涵盖项目简介、系统架构、快速开始指南等
- REPORT.docx/REPORT.pdf 课程实践报告，详细记录项目实训过程中各工作内容的设计过程及完成情况，如使用了 AI 辅助开发，须在报告中明确说明所使用的 AI 模型及辅助完成的具体工作
- 5-10 分钟的项目展示视频，发布在 Bilibili 平台，标题以【武理 26 软工实践】为前缀

（5） 协作规范

团队协作是小组项目成功的关键保障。所有小组成员须在 **GitHub** 平台上加入同一小组，共用同一代码仓库，确保代码基线的统一性。小组倡导开放、透明的沟通文化，鼓励成员之间及时反馈问题和分享开发经验。在代码评审环节，小组积极采用同行评审与结对编程等实践方式，通过交叉审查发现潜在缺陷，同时促进团队成员之间的知识传递与技术成长。每位成员在开发过程中应保持对主分支稳定性的高度敏感，在进行功能合并前充分测试个人代码，避免因个人提交影响团队整体进度。通过以上协作规范的有效执行，小组力求在项目实施过程中形成高效、有序、互信的团队工作氛围。

3. 小组软件过程

3.1 代码仓库

3.1.1 仓库基本信息

本项目采用 **GitHub** 作为小组协同开发平台，项目仓库为：

<https://github.com/wutcst/kaifaluckyginger.git>

仓库中的项目为基于经典 **worldofzuul** 扩展而来的 **Web** 冒险游戏，技术栈包括 **Java**、**Maven**、**HTML**、**CSS**、**JavaScript**、**MySQL/JDBC**、**Docker** 与 **Render**。项目不仅保留了 **Zuul** 原有的命令式冒险结构，还扩展了 **Web** 图形界面、用户登录注册、游戏状态保存读取、房间地图、角色动画、物品系统、谜题流程和云端部署能力。

3.1.2 仓库目录结构

仓库采用较清晰的分层目录结构，主要包括：

src/main/java/cn/edu/whut/sept/zuul/：Java 后端与游戏核心代码，包括 **Game**、**Room**、**Player**、**Item**、各类 **Command**、**GameController**、**GameWebServer**、**GameStateManager**、**DatabaseManager** 等。

src/test/java/cn/edu/whut/sept/zuul/：测试代码，覆盖房间、玩家、物品、命令、保存读取、通关检测等核心功能。

web/：前端页面和交互逻辑，包括 **index.html**、**style.css**、**game.js**。

web/assets/：游戏静态资源，包括房间背景、角色帧动画、物品图标、地图、谜题图片、音乐和 **UI** 资源。

.github/workflows/：GitHub Actions 自动化检查配置。**pom.xml**：Maven 项目配置，包含 **Java** 版本、编码、依赖和打包配置。

Dockerfile、**render.yaml**：项目容器化和 **Render** 云端部署配置。

README.md、GAMEPLAY.md：项目说明、玩法指南、运行方式和测试建议文档。

📁 .github/workflows	ci(actions): add frontend syntax check	2 weeks ago
📁 lib	feat(persistence): add MySQL connector dependency	last month
📁 src	feat(database): support managed MySQL environment config...	last week
📁 web	fix(log): suppress all save failure messages	last week
📄 .dockerignore	build(deploy): containerize Java backend for Render	last week
📄 .gitignore	chore(gitignore): add comprehensive .gitignore for build, IDE ...	last month
📄 Dockerfile	build(deploy): containerize Java backend for Render	last week
📄 GAMEPLAY.md	feat/assets): Add puzzle images for computer and notebook	3 weeks ago
📄 README.md	docs(deploy): add Render and TiDB Cloud deployment guide	last week
📄 pom.xml	build(deploy): package executable backend jar	last week
📄 render.env.example	docs(deploy): add Render and TiDB Cloud deployment guide	last week
📄 render.yaml	fix(deploy): use Render free instance plan	last week

3.1.3 仓库文档与可追踪性

 README

 **World of Zuul: Campus Adventure**

一款基于 Java + Web 的校园探索解谜游戏。项目从经典 *World of Zuul* 文本冒险出发，扩展为带有角色动画、房间地图、物品交互、谜题流程、登录注册、保存读取和后端 API 的完整浏览器游戏体验。

JAVA8+MAVENBUILDWEBHTML | CSS | JSMySQLPERSISTENCESTATUSPLAYABLE

 **项目简介**

World of Zuul: Campus Adventure 是一个以大学校园为舞台的 Web 冒险游戏。玩家从校园入口出发，在主大厅、图书馆、计算机实验室、森林小路、上锁宝库、传送房间等场景中探索，收集关键物品，破解线索谜题，最终获得 `final_key` 并完成逃离。

项目保留了 Zuul 原本的面向对象结构和命令式交互，同时加入了完整的 Web 可视化体验：

- 🖼️ 房间背景图与校园地图展示
- 👤 四方向角色行走动画
- 👉 可达方向按钮提示与键盘移动
- 🎒 背包、物品详情、重量和使用效果
- 📓 笔记本、电脑、密码箱等谜题流程
- 👤 登录注册、保存读取、状态同步
- 🌐 Java 后端 HTTP 服务与前端 API 通信

仓库通过 README.md 说明项目背景、核心特色、系统架构、项目结构、运行方式和部署方式；通过 GAMEPLAY.md 补充玩法规则、命令说明、物品系统、负重系统、返回命令、传送房间等细节。开发过程中的功能提交、修复提交、测试提交、文档提交和部署提交均保留在 Git 历史中，便于教师根据分支、提交记录、Pull Request 和 Actions 结果追踪每位成员的工作内

容。

3.2 软件版本计划

3.2.1 版本规划原则

本项目按照“核心功能优先、交互体验逐步增强、最后完善测试与部署”的方式推进。小组在开发前根据任务要求拆分功能点，并结合 **GitHub** 里程碑、Issue、分支和 **Pull Request** 管理开发过程。版本计划不是一次性完成全部功能，而是将项目划分为核心模型、Web 交互、保存读取、测试文档和部署发布等阶段，逐步集成和验证。

3.2.2 阶段性版本安排

第一阶段为核心游戏模型阶段。该阶段重点完成房间拓扑、玩家模型、物品模型、命令解析、房间移动、物品拾取与丢弃、背包负重、返回命令、传送房间、锁定房间和通关检测等基础逻辑，为后续 Web 前端和保存系统提供稳定的数据模型。

第二阶段为 Web 基础交互阶段。该阶段在 Java 后端基础上增加浏览器界面，完成登录注册、开始菜单、方向控制、命令输入、日志反馈、房间状态展示、角色动画和前后端状态同步，使项目从命令行程序扩展为可视化 Web 游戏。

第三阶段为功能扩展与体验优化阶段。该阶段继续完善场景物品拾取气泡、背包与负重进度条、交互式地图、三槽存档、宝库解锁、传送确认、谜题弹窗、物品详情和方向提示等功能，并修复移动冲突、地图标记偏移、存档覆盖、进度不同步等问题。

第四阶段为测试与代码规范完善阶段。该阶段由负责测试和质量保障的成员补充核心类测试、命令测试、回归测试和 Javadoc 注释，并通过 **Pull Request** 检查代码风格、接口一致性、注释完整性和功能正确性。

第五阶段为集成与发布阶段。该阶段完成 **Maven** 打包、**GitHub Actions** 自动检查、**Docker** 容器化、**Render** 前后端部署、**TiDB Cloud/MySQL** 环境变量配置和部署文档编写，使项目具备从代码提交到自动检查、打包和云端运行的完整流程。

3.2.3 里程碑管理

小组使用 **GitHub** 里程碑记录阶段目标和版本节点。每个阶段围绕明确的功能目标推进，例如核心模型完成、Web 交互完成、存档与地图完成、测试补充完成、部署发布完成等。里程碑用于控制整体进度，Issue 和 **Pull Request** 用于记录具体任务和代码变更，最终通过仓库历史体现各成员的实际贡献。

Open	0	Closed	6	Sort
M6 项目最终稳定性收尾、测试与部署				...
本里程碑用于完成项目最终交付前的稳定性收尾工作，重点包括单元测试补充、代码规范化检查、必要注释完善、前后端功能回归测试、README 与项目报告内容整理，以及最终部署上线准备。本阶段不再进行大规模功能重构，主要围绕已有 Java + Web 版 Worl...				100% complete 0 open 4 closed
Due by June 12, 2026 • 4/4 issues closed				
M5 Web 游戏设计与稳定性优化升级				...
在前期核心框架的基础上，本里程碑对游戏进行了大幅度的功能扩展与整体体验升级，重点完成了以下模块的集成与优化：- 全新谜题与通关系统：引入笔记本、电脑、密码箱等交互谜题，重构通关条件，增加胜利结局及多种反馈弹窗，丰富游戏可玩性与叙事深...				100% complete 0 open 19 closed
Due by June 9, 2026 • 19/19 issues closed				
M4 Web 可视化交互与完整游戏体验实现				...
实现房间背景的切换，可移动区域以及路径规划，人物移动方向，最终达到完整的前端游戏交互体验。				100% complete 0 open 7 closed
Due by May 31, 2026 • 7/7 issues closed				
M3 游戏扩展与体验增强				...
完成游戏体验增强相关开发，包括 2D 可视化场景资源整理、角色与道具静态资源准备、Phaser.js 前端原型接入、房间场景切换、玩家移动展示、背包与交互界面优化，使项目从文本/Web 交互逐步扩展为具有完整画面表现的 2D Web 游戏。				100% complete 0 open 1 closed
Due by May 25, 2026 • 1/1 issues closed				
M1 项目分工与基础模块开发				...
完成小组分工、Issue 创建、功能分支建立，以及核心玩法、命令交互、Web 服务、前端界面的基础开发。				100% complete 0 open 4 closed
Due by May 20, 2026 • 4/4 issues closed				
M2 命令解析与玩家交互命令基础开发				...
完成命令解析器框架和所有玩家交互命令的开发与测试。具体包括：设计命令框架和输入解析器，实现查看房间、返回上一房间、拾取物品、丢弃物品、使用物品、查看物品列表、吃特殊物品、保存游戏、加载游戏、查看游戏进度、帮助信息以及退出游戏等功能...				100% complete 0 open 2 closed
Due by May 20, 2026 • 2/2 issues closed				

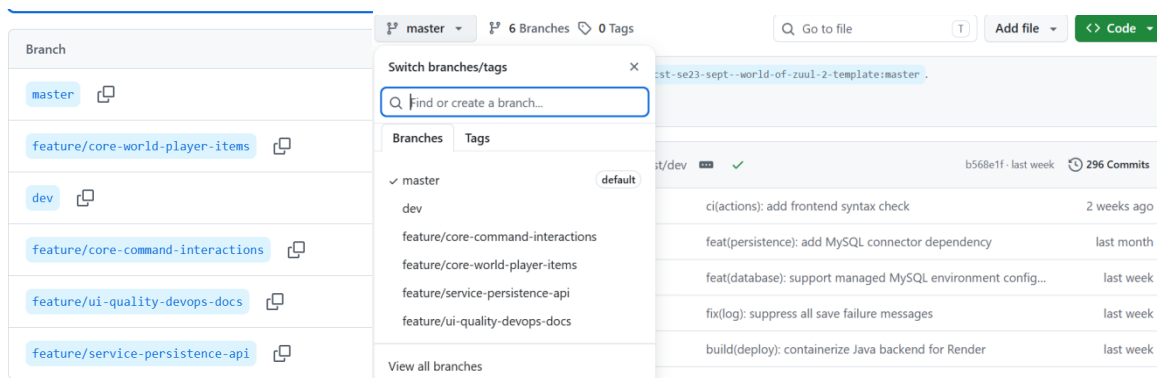
3.3 开发分支模型与提交流程

3.3.1 分支模型设计

项目采用以 master、dev 和 feature/* 为核心的分支模型：

- **master**：稳定主分支，用于保存相对完整、可运行、可发布的版本。
- **dev**：开发集成分支，用于合并各成员功能分支并进行阶段性联调。
- **feature/coreworldplayeritems**：核心世界、玩家、物品、房间、前端资源和部分部署相关功能分支。
- **feature/corecommandinteractions**：命令交互、移动控制、物品命令和部分前端交互优化分支。
- **feature/servicepersistenceapi**：后端服务、用户会话、保存读取和持久化接口相关分支。
- **feature/uiqualitydevopsdocs**：界面质量、文档、部分交互说明和工程化辅助相关分支。
- **integration/***：临时集成检查分支，用于在合并到主开发分支前进行本地预览、冲突处理或多分支合并验证。

这种分支模型能够保证每位成员在个人功能分支上独立开发，同时通过 dev 分支进行阶段性集成，避免所有成员直接修改主分支造成冲突。



3.3.2 提交流程

小组成员开发前先同步远程仓库最新代码，再在自己的功能分支上进行开发。完成一个相对独立的功能或修复后，按照约定提交代码，并将分支推送到 GitHub。提交后通过 Pull Request 将功能分支合并到 dev 或 master。

基本流程如下：

1. 从远程仓库同步最新代码。
2. 切换到个人功能分支进行开发。
3. 本地完成编译、运行或页面验证。
4. 按功能点提交代码。
5. 推送个人分支到 GitHub。
6. 创建 Pull Request。
7. 由其他成员进行代码检查和功能确认。
8. GitHub Actions 自动执行语法检查和 Maven 检查。
9. 检查通过后合并到目标分支。
10. 合并后其他成员同步最新代码继续开发。

3.3.3 提交说明规范

提交说明采用较清晰的类型前缀，便于从 Git 历史中识别提交目的。常见类型包括：

- feat: 新增功能，例如新增物品气泡、三槽存档、地图、登录注册等。
- fix: 修复问题，例如修复存档覆盖、移动冲突、方向提示、背包负重不同步等。
- test: 增加或调整测试，例如核心类测试、命令测试、回归测试等。

- docs: 文档修改, 例如 README、玩法说明、部署文档、Javadoc 注释等。
- ci: 持续集成配置, 例如 GitHub Actions 工作流。
- build: 构建和打包配置, 例如 Maven Shade、Dockerfile。
- style: 样式或界面表现调整。
- chore: 项目配置、清理临时文件、维护性修改。

通过规范化提交说明, 教师和组员可以快速判断每次提交对应的是功能开发、缺陷修复、测试补充还是工程配置。

feat(assets): add background music file	364bd32	<>
style(game): add styles for music control panel	8f410e6	<>
feat(game): add music button and volume control panel	0581ae6	<>
feat(game): persist music state in localStorage	c14597d	<>
fix(game): 修复吃饼干功能及物品命令后端冲突	1638507	<>
fix(game): 修复拾取/丢弃物品后背包负重不更新	1452f57	<>
Merge pull request #52 from wutkst/feature/core-world-player-items	06a3018	Verified <>
fix(deploy): use Render free instance plan	5738b6c	<>
Merge pull request #51 from wutkst/feature/core-world-player-items	4ae6938	Verified <>
fix(deploy): correct Render blueprint YAML syntax	717ceec	<>

3.4 代码规范与检查流程

3.4.1 Java 代码规范

Java 代码遵循面向对象设计原则, 尽量做到类职责清晰、方法功能单一。核心游戏逻辑由 Room、Player、Item、Game 和各类 Command 承担; Web 请求处理由 GameWebServer 和 GameController 承担; 保存读取由 GameStateManager 和 DatabaseManager 负责。通过这种分层方式, 避免所有逻辑集中在单一类中, 提升代码可读性和可维护性。

项目统一使用 UTF-8 编码, 并在 pom.xml 中配置:

project.build.sourceEncoding 为 UTF-8

maven.compiler.source 为 8

maven.compiler.target 为 8

这样可以减少中文注释、中文提示和跨平台编译时的乱码问题。

3.4.2 前端代码与资源规范

前端代码集中在 `web/game.js`、`web/style.css` 和 `web/index.html` 中。前端逻辑按照功能拆分为房间配置、角色移动、状态同步、背包渲染、物品拾取、地图弹窗、存档读取、模态框控制等模块化函数，避免将所有交互写成无结构的事件堆叠。

静态资源按照类型归档：

- **characters**：角色精灵图和四方向行走帧。
- **items**：物品图标和剧情物品资源。
- **rooms**：各房间背景图。
- **maps**：校园地图资源。
- **puzzles**：谜题相关图片。
- **ui**：界面辅助素材。
- **audio**：背景音乐资源。

资源分类管理有利于后期替换素材、定位引用关系和减少重复文件。

3.4.3 注释与文档规范

项目中对核心类、服务类、控制器方法和状态管理方法补充了 Javadoc 注释。例如提交历史中包含对 `GameController`、`GameStateManager`、`Game` 等类的注释完善。注释重点说明类职责、方法用途、参数含义和返回结果，而不是简单重复代码语句。

代码注释由相关负责成员补充，Pull Request 检查时会关注关键类和复杂方法是否具有必要说明。对于接口、保存读取、命令执行、状态同步等容易产生理解成本的部分，要求通过注释或文档降低维护难度。

3.4.4 Pull Request 人工检查流程

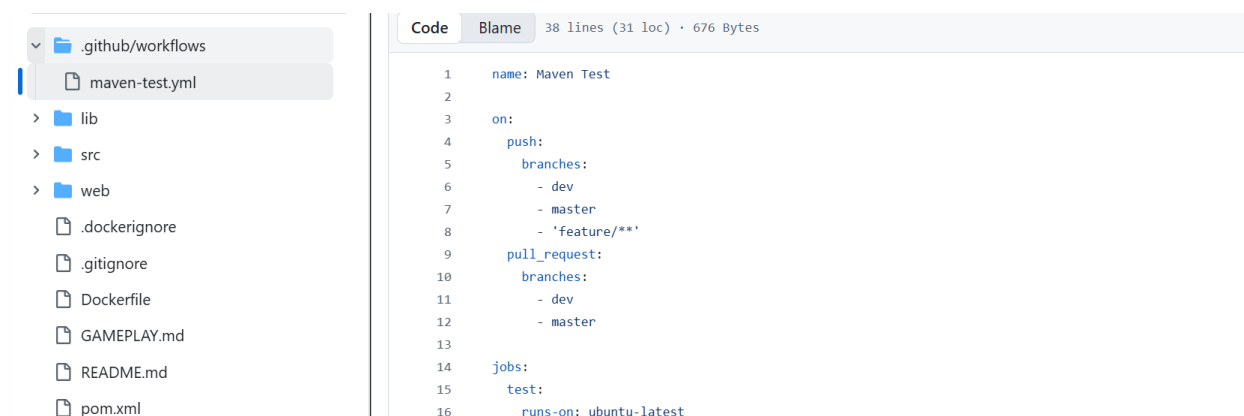
每个 Pull Request 提交后，小组会安排其他同学进行代码检查。检查内容包括：

1. 功能是否符合 Issue 或里程碑目标。
2. 是否影响其他成员负责的模块。
3. 是否存在明显命名不规范、重复代码或无关文件。
4. 是否包含必要注释和说明。

5. 是否能够通过本地运行或页面验证。
6. 是否会引入合并冲突或破坏已有功能。
7. 是否符合提交说明和分支管理规范。

人工检查通过后，再结合 **GitHub Actions** 的自动检查结果决定是否合并。对于存在问题的 **PR**，先由提交者修改，再重新检查。

3.4.5 自动化检查流程



仓库配置了 **GitHub Actions** 工作流 `.github/workflows/maven-test.yml`。该工作流在以下情况下触发：

向 `dev`、`master` 或 `feature/**` 分支 `push` 代码。

向 `dev` 或 `master` 发起 `Pull Request`。

自动检查步骤包括：

1. 拉取仓库代码。
2. 配置 `JDK 8`。
3. 配置 `Node.js 20`。
4. 执行 `node --check web/game.js` 检查前端 `JavaScript` 语法。
5. 执行 `mvn test` 检查 `Java` 项目编译与测试流程。

通过人工检查和自动化检查结合，项目能够在合并前发现语法错误、构建错误和部分功能回归问题。

3.5 项目测试计划与实施流程

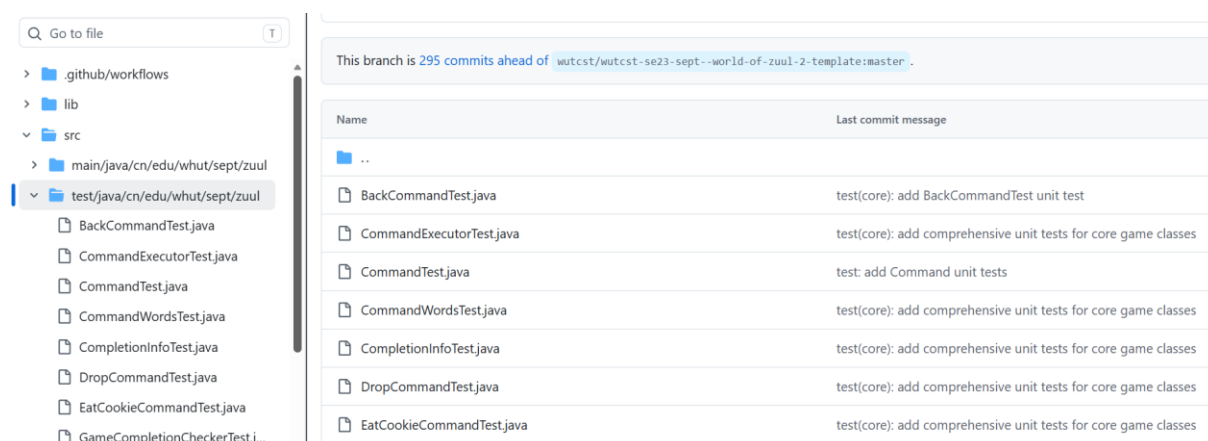
3.5.1 测试目标

项目测试目标是保证核心游戏逻辑、命令交互、**Web API**、保存读取和前端页面在多次迭代后仍能正常工作。由于项目涉及后端状态、前端交互、

数据库持久化和游戏资源展示，因此测试计划同时包含自动化测试和人工功能验证。

3.5.2 自动化测试范围

仓库 `src/test/java/cn/edu/whut/sept/zuul/` 下包含多类测试文件，主要覆盖以下内容：



房间模型测试：RoomTest、RoomExtendedTest、TransporterRoomTest、LockedRoomTest

玩家与物品测试：PlayerTest、ItemTest、ItemExtendedTest

命令测试：GoCommandTest、BackCommandTest、TakeCommandTest、DropCommandTest、UseCommandTest、ItemsCommandTest、LookCommandTest、SaveCommandTest、LoadCommandTest、StatusCommandTest 等

解析与工具测试：ParserTest、CommandTest、CommandWordsTest、JsonUtilTest、LoggerTest

游戏流程测试：GameCompletionCheckerTest、CompletionInfoTest、GameRegressionTest

状态管理测试：GameStateManagerTest

这些测试用于验证核心类行为、命令执行结果、物品和背包状态、通关信息、保存读取流程以及回归场景。

3.5.3 前端与接口测试

前端测试主要包括两类：

第一类是语法检查。GitHub Actions 中执行 `node --check web/game.js`，用于检查前端脚本是否存在语法错误，避免因 JavaScript 语法问题导致页

面无法加载。

第二类是功能验证。小组成员在浏览器中手工验证登录注册、开始菜单、方向移动、物品拾取气泡、背包负重、物品详情、交互地图、三槽存档、传送确认、谜题弹窗和音乐控制等功能。对于涉及前后端通信的功能，需要同时观察页面反馈和后端接口返回结果。

3.5.4 测试实施流程

项目测试流程分为开发自测、PR 检查、集成测试和发布前验证四个阶段。

开发自测阶段，成员在个人分支上完成代码后，先运行本地项目，确认新增功能能够正常使用。例如新增物品拾取功能后，需要验证房间物品是否消失、背包是否增加、负重是否变化、日志是否输出。

PR 检查阶段，成员提交 Pull Request 后，由其他同学检查代码，并通过 GitHub Actions 执行 `node --check web/game.js` 和 `mvn test`。如果检查失败，不能直接合并，需要修复后重新提交。

集成测试阶段，多个功能分支合并到 dev 后，小组重点检查跨模块功能。例如命令系统、背包状态、保存读取和前端显示之间是否同步；登录注册后是否能正确进入游戏；读取存档后房间、物品和进度是否一致。

发布前验证阶段，在合并到稳定分支或部署到 Render 前，小组会进行冒烟测试，确认首页可访问、后端健康检查可用、基础游戏流程可执行、存档和读取无明显异常。

3.5.5 缺陷修复闭环

测试过程中发现的问题通过修复提交解决，并保留在 Git 历史中。例如提交记录中包含修复吃饼干与物品命令冲突、修复拾取/丢弃后背包负重不更新、修复存档物品丢失、修复移动边界卡住、修复地图标记偏移、修复无效方向提示等内容。每个缺陷修复后会重新进行相关功能验证，形成“发现问题—提交修复—重新检查—合并”的闭环。

3.6 项目集成与发布规范

3.6.1 集成原则

项目集成遵循“先功能分支开发，再合并到开发分支，最后进入稳定分支”的原则。个人功能分支只负责各自模块的开发，dev 分支承担阶段集成，master 分支保留稳定版本。对于可能引发较大冲突的功能，小组会使用 integration/* 分支进行临时合并测试，确认没有明显冲突后再进入主开发分支。

3.6.2 Maven 构建与打包

项目使用 Maven 管理 Java 工程。pom.xml 中配置了 MySQL JDBC 依赖, 并使用 maven-shade-plugin 在 package 阶段生成可执行 Jar。打包时会将主类设置为:

```
cn.edu.whut.sept.zuul.WebMain
```

因此项目可以通过 Maven 构建为可执行后端程序, 便于本地运行和云端部署。常用命令包括:

```
bash
```

```
mvn test
```

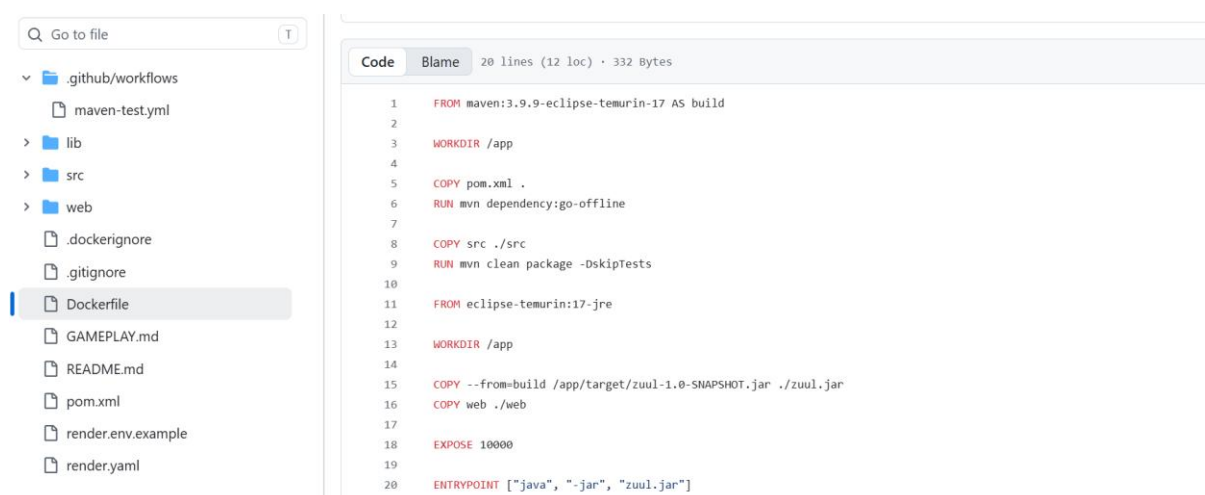
```
mvn clean package
```

3.6.3 Docker 容器化发布

项目提供 Dockerfile 用于后端服务容器化。Docker 构建流程采用多阶段构建:

1. 使用 Maven 镜像下载依赖并执行打包。
2. 使用 JRE 镜像运行生成的 zuul.jar。
3. 将 web 静态资源复制到容器中。
4. 通过 java -jar zuul.jar 启动服务。

这种方式可以减少部署环境差异, 使后端服务在 Render 等平台上以统一方式运行。



3.6.4 Render 前后端部署

项目使用 render.yaml 描述云端发布配置, 包含两个服务:

zuul-api: 后端 Web 服务，采用 Docker 运行，配置健康检查路径 /api/health。

zuul-web: 静态前端服务，发布 web 目录。



前端构建时通过环境变量 `API_BASE_URL` 生成 `web/config.js`，使前端能够连接部署后的后端接口。后端通过环境变量读取数据库配置，包括：

- `DB_JDBC_URL`
- `DB_USER`
- `DB_PASSWORD`
- `DB_CREATE_DATABASE`

数据库连接信息不直接写入代码，避免敏感信息进入仓库。

3.6.5 发布前检查规范

在发布前，小组需要完成以下检查：

1. dev 分支功能已完成阶段集成。
2. Pull Request 已经过成员检查。
3. GitHub Actions 自动检查通过。
4. 本地能够正常启动 Web 游戏。
5. 前端主要页面能够访问。
6. 登录注册、移动、拾取、存档、读取、地图和传送等核心流程能够运行。
7. Render 后端健康检查可用。
8. 数据库环境变量已正确配置。
9. README 和部署文档已更新。

3.6.6 发布成果

最终项目形成了较完整的发布形态：本地可通过 Maven 和 Java 启动，云端可通过 Docker 与 Render 部署，数据库可通过 MySQL/TiDB Cloud 持久化保存用户和游戏状态。同时，GitHub Actions 提供提交和 PR 阶段的自动检查，README、GAMEPLAY 和部署文档提供运行、测试、玩法和发布说明，使项目具备较规范的软件工程实践流程。

4. 小组成员任务报告

4.1 成员一（刘紫媛）任务分工及实施过程描述

4.1.1 工作内容概述

我负责项目从核心后端、前端交互、视觉资源到工程化部署的完整开发链路。后端方面，完成游戏世界与房间拓扑、Player/Item 模型、物品容器、背包负重、拾取丢弃、命令解析、房间移动、多级返回、传送房间、钥匙解锁、魔法饼干、通关进度、登录注册、状态同步和存档读写等核心功能。前端方面，负责房间背景、角色动画、物品图标和校园地图等资源的查找、生成、整理与接入，设计房间入口坐标、隐藏移动路径、角色四向移动和场景切换动画，并实现开始菜单、方向控制、日志反馈、物品拾取气泡、背包负重、物品详情、交互地图、传送/谜题弹窗和三档存档等交互功能。

同时，承担里程碑建立、个人分支维护、Pull Request 协作、Readme/玩法/部署文档编写，以及 Maven、GitHub Actions、可执行 Jar、Docker、Render、TiDB Cloud 等 CI/CD 自动检查、打包与部署配置。

4.1.2 核心模型与游戏规则

扩展 World of Zuul 的面向对象基础：以 Room 构建校园房间拓扑和物品容器，以 Player 独立保存姓名、当前位置、访问记录、背包及最大负重，以 Item 封装名称、描述和重量。拾取时先累计背包重量并校验上限，成功后同步房间物品、玩家背包与收集进度；同时增强命令解析、方向移动和多层 back 回退。

1. 新增传输房间、宝库、钥匙解锁、魔法饼干及探索/收集通关规则。
2. 利用继承与状态封装实现特殊房间，将各类规则从 Game 类解耦。
3. 编写 Room、Player、Item 等核心回归测试及 GAMEPLAY 玩法文档。

4.1.3 Web 游戏框架与状态协同

基于核心模型搭建页游前端界面，实现登录注册、菜单、指令 / 方向

操控、日志、房间与背包面板；前端调用 API 同步后端游戏数据，统一渲染房间、道具、负重、进度；后端离线时支持本地预览存档，保障基础游玩连贯。

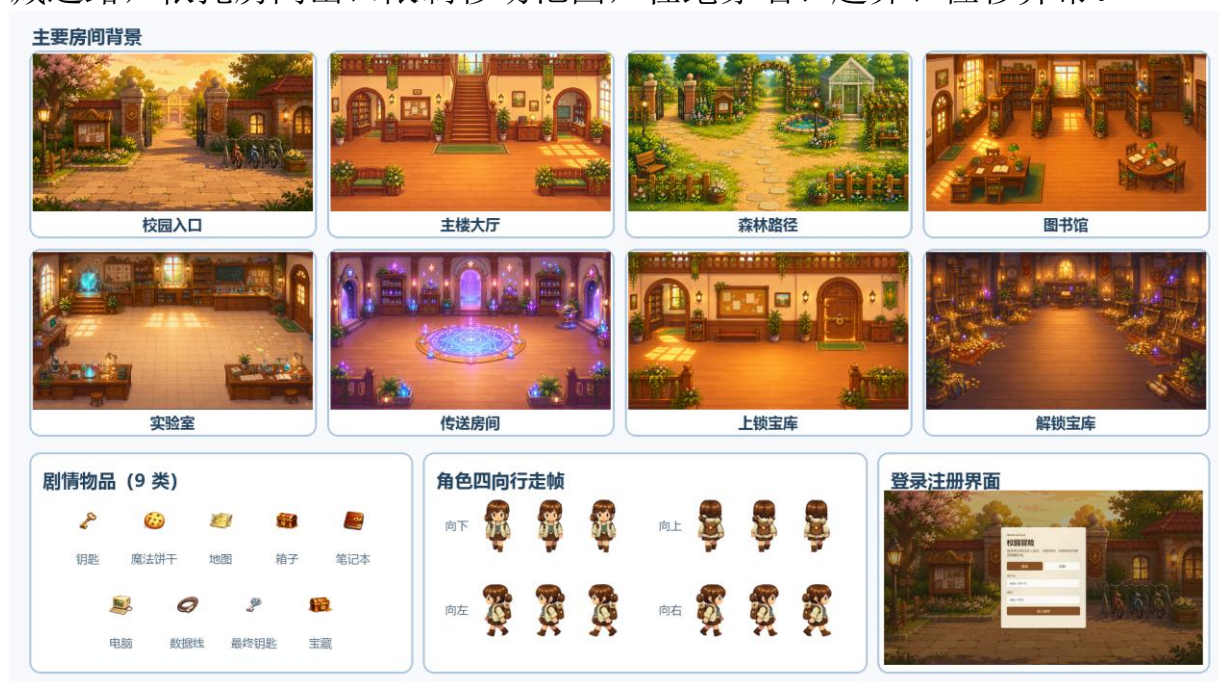
工作板块	代表性实现	主要位置
核心领域	房间拓扑、玩家背包、物品重量、特殊房间	Room.java、Player.java、Item.java
命令规则	移动、拾取、丢弃、查看、回退、使用物品	Parser、Command、GoCommand
Web 协同	认证、命令 API、状态同步、中文交互	web/game.js、GameWebServer.java

4.1.4 可视化交互与重点功能

我负责将原有命令行文本冒险扩展为具备完整交互界面的 2D 校园冒险游戏。基础界面包括登录注册、开始菜单、场景显示、方向控制、命令输入、运行日志、房间状态和背包面板，并通过前后端接口同步玩家位置、当前房间、房间物品、背包、负重和游戏进度等状态。

前端方面，统筹全量视觉素材制作、分类与加载，为各校园专属场景配置独立背景，建立房间标识、出入口与前端场景映射；

后端方面，实现角色四向行走逐帧动画，配置各房间底层行走路线，隐藏通路，依托房间出口限制移动范围，杜绝穿墙、越界、位移异常。



4.1.5 游戏功能扩展与交互体验优化

基础框架搭建完毕后，继续迭代优化物品交互、特殊房间、地图、背包及存档模块，完善交互反馈与游戏进度管理体系。

(1) **场景物品拾取交互**：为场景道具添加拾取气泡，优化图标视觉布局，支持点击拾取物品并同步更新场景、背包等数据并自动存档。

(2) **背包负重体系优化**：优化背包面板，完善物品详情、排序等功能，新增负重可视化进度条，支持切换三种状态，提供可视化反馈。

(3) **宝藏房间权限管控**：实现宝藏房间钥匙准入机制，核验道具权限管控进出，解锁后更新房间背景与出口，新增专属奖励物品。

(4) **传送房间机制完善**：完善传送交互流程，玩家确认后随机跳转校园房间，推送传送提示，同步更新玩家位置、场景等游戏状态。

(5) **交互式校园地图开发**：制作可缩放校园地图弹窗，实时标注玩家坐标，调试地图点位，直观展示场景结构，助力玩家探索路线。

(6) **三槽存档系统迭代**：搭建三组存档槽，支持存档新建、覆盖、读取、删除，卡片展示游玩核心数据，后端离线可调用本地存档。

(7) **全局游玩稳定性优化**：修复位移、动画、进度异步等游戏 bug，统一弹窗交互规则，弹窗开启暂停角色移动，优化交互体验。



4.1.6 测试、持续集成与部署

除功能开发外我进一步完善工程化链路：配置 UTF-8 与 Java8 兼容，搭建 Maven、JS 语法校验；借助 Maven 插件打包可执行 Jar，编写 Docker 部署配置，适配动态端口、健康检查、API 与数据库环境变量，搭建提交-校验-打包-云端部署 DevOps 流程。

工程环节	完成内容	对应配置
自动检查	push/PR 触发 Maven 构建与前端语法检查	.github/workflows/maven-test.yml
可执行交付	Shade 打包后端 Jar, Docker 容器化运行	pom.xml、Dockerfile
云端部署	Render 前后端服务、健康检查和 API 注入	render.yaml、web/config.js
云数据库	JDBC 地址、账号和密码通过环境变量注入	DatabaseManager.java

4.1.7 工作小结

本次开发中，我以统一的房间、玩家、物品和游戏状态模型为基础，使后端规则、前端交互、存档数据与界面反馈保持一致。在实现过程中，我注重类职责划分、资源分类管理、配置复用和异常场景处理，并通过规范化提交、个人功能分支、Pull Request、自动检查及部署验证保障代码质量和开发过程的可追溯性。

通过持续迭代，我能够根据实际运行结果定位并修复状态不同步、移动冲突、存档覆盖和交互反馈等问题，使项目由“功能可运行”逐步达到“逻辑完整、交互稳定、便于维护和部署”的状态。

4.2 成员二（殷瑞涵）任务分工及实施过程描述

4.2.1 任务概述

首先，我主要负责后端核心命令系统的设计与实现，采用 Command 模式对游戏指令进行抽象封装，并完成了各业务 Command 类开发与逻辑闭环。

在场景交互层面，我设计并实现了前端十字通道移动系统，通过通道吸附与两级出口检测，实现了玩家在场景中的平滑自由移动与房间切换。

在构造谜题系统方面，我独立设计了完整的谜题交互链路，包含密码验证弹窗、物品拖拽与点击交互逻辑、虚拟物品的动态管理，以及谜题状态的本地持久化，保障玩家操作流畅性与进度保存。

在前后端协同层面，我对 syncFromBackendStatus 进行了改造，引入 allowRoomChange 机制控制房间切换权限，并完成背包物品合并策略的优化，确保双端状态一致性。

对于系统完善与问题修复，不仅实现了完整的游戏状态保存与恢复功能，支持三种进入方式的差异化处理，并设计了自动存档策略以提升容错性；同时解决了物品异常丢失、角色卡死、丢弃物品错误返回、返回上一房间逻辑紊乱、删除按钮失效等关键问题，显著提升了游戏稳定性。

最后我优化了全流程游戏体验，覆盖胜利流程、行走状态禁止拾取、传送逻辑优化以及背景音乐系统集成等功能以及实现。

4.2.2 实施过程描述

(1) 后端核心命令交互系统

实施项目初期，我首先负责搭建后端命令交互的底层框架。设计上采用命令模式，将每个游戏操作封装为独立的命令类，统一实现 `CommandExecutor` 接口，该接口定义 `execute` 方法返回布尔值表示是否触发退出。`Command` 类封装命令词和第二参数，`Parser` 负责将用户输入解析为 `Command` 对象，处理大小写不敏感和多余空格裁剪，并针对 `eat` 命令做了多词参数的特殊处理。

各业务 `Command` 类涵盖了方向移动、房间历史回退、负重检测、物品拾取与回归初始房间、钥匙开锁与地图查看、负重上限提升、房间详情展示、进度查看以及存档读档等核心操作。后期为适配前端需求，在 API 层为物品增加了 `itemType`、`useEffect`、`usable` 三个字段，使前端能根据物品类型动态生成交互按钮。

(2) 十字通道移动系统

实施项目初期，我还设计并实现了前端十字通道移动系统。建立了房间配置体系，为每个房间定义中心坐标、通道类型及移动边界，不同房间配置不同的通道组合。实现了通道吸附算法，仅垂直通道时将 `x` 轴吸附至中心，仅水平通道时将 `y` 轴吸附至中心，双向通道则根据移动方向自动吸附，确保玩家始终在有效路径上移动。步进计算函数根据通道类型过滤无效方向，移动后裁剪越界坐标，保证移动连续合理。出口检测采用落点检测和穿越检测两级机制，确保无论步长大小都能正确触发房间切换。

移动速度方面实现了键盘按住加速机制，通过追踪按键状态和按住时长返回对应步长。此外为校园酒吧最左侧出口实现了边界触发检测，当玩家移动至边界时自动弹出出口确认弹窗。

(3) 构建完整谜题系统

实施项目中期，我详细设计并实现了完整的谜题系统。谜题链路为，`key` 解锁宝库获得 `treasure`，`treasure` 打开 `notebook` 展示谜题图片，据此输入电脑密码 1235，连接 `cable` 后展示第二张谜题图片，输入箱子密码 1768 获得 `final_key`，最终打开出口大门。密码仅两个且存于 `localStorage` 以减少后端改动。具体实现了密码验证弹窗和谜题图片展示弹窗，支持 4 位数字输入和异步回调验证，替代了原有的阻塞式 `prompt` 输入。

物品交互逻辑完整重写了 `showItemDetail`，根据物品类型和谜题状态动态生成操作按钮，`key`、`treasure`、`computer`、`cable`、`box`、`final_key` 各自拥有独立的交互处理逻辑。前端虚拟物品由 `gameState` 独立管理，在 `buildFullState` 和 `applyFullState` 中统一保存恢复。谜题状态通过 `notebookUnlocked`、

computerStarted、boxOpened 等布尔字段管理，每次变化时自动保存至 localStorage。

（4）前后端状态同步改造

实施于阶段中后期，是项目中最复杂的问题。原始设计中，前端每次与后端交互后都会调用 syncFromBackendStatus，用后端数据完整覆盖前端状态，但后端并不感知前端虚拟物品和谜题状态，导致频繁出现物品丢失。

改造时首先引入 allowRoomChange 机制，默认为 false 时仅更新房间描述和出口信息，不覆盖 currentRoomId、inventory 和 roomItems，仅在初始加载、新游戏和读档时设为 true。其次优化了背包合并策略，改为前端优先，使用扩展运算符合并前后端背包，确保虚拟物品不被覆盖。同时调整了物品命令后的刷新逻辑，take、drop、use 等物品命令后不再调用 refreshGameStatus，避免后端旧数据覆盖前端最新状态。此外还修复了 startGameFromMenu 加载存档时的竞态条件问题。

（5）系统完善与 Bug 修复

实施阶段为项目全周期，涵盖存档系统完善与各类关键问题的修复。存档系统方面实现了完整的游戏状态保存与恢复功能，覆盖当前房间、玩家坐标、背包物品、各房间物品、全部谜题状态、roomHistory 历史栈及已访问房间集合。三种进入方式采用差异化处理，新游戏时重置全部状态，读取存档时从服务器加载恢复，进入当前进度时从本地加载上次离开的状态。自动存档机制在 takeItem、dropItem、useItem、eatCookie、enterNextRoom 等关键操作后均触发保存。

Bug 修复方面也解决了多个关键问题，包括通过 allowRoomChange 机制修复物品丢失，通过 load 路径不再调用 enterGameFromAuth 修复读档物品丢失，通过关键操作后自动保存修复进度丢失，将步长从 12 改为 9 并新增穿越检测修复房间交界卡住，新增 ITEM_HOME_ROOM 映射使丢弃物品回到初始房间，将 back 逻辑移至 handleCommand 并持久化 roomHistory 修复返回失效，以及修复删除按钮不显示和后端传送到 locked_room 等问题。

（6）游戏体验优化

实施项目中后期完成优化，覆盖胜利流程、行走禁止拾取、传送优化和背景音乐系统。胜利流程实现了 showWinModal 弹窗，通关后返回菜单。行走禁止拾取在 takeItem 中增加 isMoving 检测，防止移动中误触拾取。传送优化方面前端过滤合法目标，排除 teleport_room 和宝库，将玩家传送到目标房间 center 位置。校园酒吧出口在 movePlayerStep 中增加边界检测，触发 showExitDialog 弹窗。另外，新增了背景音乐系统，在标题栏新增音乐控

制按钮，支持开关切换和音量滑块，状态通过 localStorage 持久化。

4.2.3 最终成果总结

经过团队成员的共同努力与持续迭代，我们最终交付了一套功能完整、运行稳定的校园解密冒险游戏系统。后端方面，我基于命令模式构建了高内聚低耦合的命令交互框架，所有游戏指令被抽象为独立的命令类，通过统一的执行接口和解析调度机制，实现了灵活的指令扩展与维护。场景交互方面，构建了基于十字通道的可视化移动系统，实现了玩家在场景中的平滑自由移动与房间切换。前端方面，完成了完整的谜题交互链路，密码验证弹窗、物品操作面板和虚拟物品管理共同构建了流畅的玩家体验。前后端状态同步机制经过深度改造，有效解决了数据覆盖和物品丢失等核心痛点，保障了双端数据一致性。存档系统的完善和各类关键 Bug 的修复显著提升了游戏的稳定性和容错能力，胜利流程、传送机制及背景音乐等体验优化则进一步增强了游戏的沉浸感和可玩性。

整个开发过程中，从底层框架搭建到上层业务实现，从前后端联调到细节体验打磨，形成了完整的技术闭环。我也不断进行测试，保证系统最终能具备稳定的可运行状态和良好的用户体验。

4.3 成员三（赵欣然）任务分工及实施过程描述

4.3.1 任务概述

我在本次小组协同开发项目中承担了前端与质量保障方向的主要开发工作，对应的开发分支为 feature/uiqualitydevopsdocs。根据小组任务分工，我主要负责 Web 前端页面开发、图形化用户界面（GUI）实现、静态资源整理、游戏交互逻辑完善、自动化持续集成（CI）配置以及课程实践报告编写等工作。此外，我还承担了部分核心功能 Bug 修复和跨分支协作任务。

我的工作贯穿了软件工程实践二的多个考察维度，包括软件架构设计（前端交互层实现）、小组软件过程与项目管理（Issue 跟踪与任务闭环）、软件分支管理（feature 分支开发与 PR 合并）、软件代码评审（通过 Pull Request 进行代码审查）以及持续集成与部署（GitHub Actions 配置与自动化检查），较好地覆盖了课程要求的各个实践环节^[10]。

4.3.2 实施过程

（1）Web 前端页面与 Java 图形化界面开发

我完成了 web/index.html 页面主体结构的搭建，以及 web/style.css 页面布局与基础样式的设计。在 web/game.js 中实现了前端基础交互逻辑，并完成了角色资源配置文件 character.json 的接入与初始化配置。该阶段的工作为后续所有前端功能的扩展奠定了结构基础。

此外，我基于 Java Swing 实现了游戏图形化界面。在 GameGUI.java 中完成了游戏窗口的基础布局设计，包括房间信息区域、物品展示区域、消息输出区域和命令输入区域。实现了方向按钮交互（North/South/East/West），能够根据当前房间出口动态更新按钮状态；实现了 look、items、back、help 等快捷命令按钮，以及 take、drop、eat cookie 等物品交互逻辑。同时实现了 System.out 输出重定向，将游戏运行信息同步展示到 GUI 消息区域。

（2） 游戏静态资源整理

为实现 2D 可视化游戏的场景展示，我与组长刘紫媛协作完成了 Web 静态资源的整理与补充。在 web/assets/ 目录下补充了校园入口、主楼、图书馆、计算机实验室等校园场景背景资源，以及钥匙、武器、NPC 角色等基础角色与道具资源，还包括铁栅栏、监狱门、木箱等场景交互 UI 素材。资源目录结构清晰、命名规范，为后续房间切换和 2D 场景升级提供了素材支持。

（3） 查看按钮交互增强与玩法说明功能实现

我实现了"查看"按钮（⊙）的交互增强功能。点击该按钮后，左侧收起的背包面板自动展开，同时底部信息面板切换到"日志"标签页。当玩家按下任意方向键或 WASD 移动时，系统自动退出"查看模式"，恢复为常规的房间信息面板。实现过程中新增了 isInspectMode 状态变量及 enterInspectMode() 和 exitInspectMode() 两个核心函数，并确保切换过程中底部按钮的激活状态能够正确同步。

为降低新玩家的上手门槛，我设计并实现了一个模态框样式的玩法说明系统。提供了三种查看入口：新游戏启动时自动弹出、菜单页面的"游戏玩法"按钮、以及游戏主界面右上角的"帮助"按钮。模态框采用半透明黑色遮罩配合白色圆角卡片的设计，支持点击遮罩层或关闭按钮两种关闭方式。玩法说明内容涵盖游戏简介、移动控制、可用命令列表、游戏目标和小提示等。

（4） 核心 Bug 修复

在游戏测试过程中发现，玩家使用 take 命令拾取物品后，右侧背包面板的负重数值和进度条不会实时更新。我定位到问题根源在于 renderInventory() 函数在拾取/丢弃后未被调用，以及重量计算条件判断存在缺陷。修复方案是在 takeItem() 和 drop 命令处理逻辑中分别添加 renderInventory() 调用，同时修正重量判断条件。修复后拾取和丢弃物品时负重数值与进度条均能实时更新。

"吃饼干"功能存在负重上限未增加、日志出现前后端矛盾信息、负重上限未持久化三个问题。我分析发现根本原因在于物品命令同时发送到后端

API 和前端本地逻辑。修复方案包括：在 `gameState` 中添加 `maxWeight` 属性；将 `eat`、`take`、`drop`、`use` 等物品命令单独抽取，不发送后端请求，直接执行前端本地逻辑；在 `buildFullState()` 和 `applyFullState()` 中分别实现 `maxWeight` 的保存与恢复。修复后吃饼干能够正确增加负重上限，日志不再出现矛盾信息，存档读档后负重上限保持正确。

针对 WASD 键移动时角色偏离预设道路、连续快速按键导致位置错乱的问题，我与殷瑞涵协作进行了修复。主要工作包括校准各房间的路径坐标点集，确保角色移动轨迹始终与画面中的道路、门口位置匹配；在移动过程中设置全局锁，移动期间忽略输入；校准所有房间的出口与入口坐标，保证切换后角色位置合理。

4.3.3 实施成果

我在本次小组协同开发项目中完成了以下主要成果：

成果类别	具体内容
Web 前端	完整的游戏页面（ <code>index.html</code> ）、样式表（ <code>style.css</code> ）和交互脚本（ <code>game.js</code> ），支持登录注册与命令交互
Java GUI	基于 Swing 的图形化游戏界面（ <code>GameGUI.java</code> ），支持方向按钮、快捷命令和物品操作
静态资源	校园场景背景、角色道具、UI 素材等分类整理的资源目录
交互功能	查看按钮增强、玩法说明模态框、WASD 键盘移动等用户体验优化
Bug 修复	背包负重更新、吃饼干功能、WASD 移动路径等关键问题修复
持续集成	GitHub Actions 自动化代码检查与测试流水线配置
课程报告	参与课程实践报告 <code>REPORT.docx</code> 的编写

上述工作均已在 GitHub 仓库中提交，对应的核心提交记录包括：

- `feat(game)`: 新增前端场景逻辑与角色配置文件
- `feat(GUI)`: 完整实现 Java 图形化游戏界面与基础交互逻辑
- `feat(assets)`: 新增校园入口、实验室、图书馆及主楼背景图片资源
- `feat(ui)`: 添加游戏玩法说明界面
- `feat(game)`: 查看按钮功能增强
- `fix(game)`: 修复吃饼干功能及物品命令后端冲突
- `fix(game)`: 修复拾取/丢弃物品后背包负重不更新

- fix: 修复 wasd 移动不了的问题

所有代码均已通过 `node check web/game.js` 语法检查和 `mvn test` 构建检查，GitHub Actions 流水线全部成功。对应的 Issue 均已闭环，功能验收通过后合并至 dev 分支。

4.3.4 总结

通过本次软件工程实践，我在 Web 前端开发、Java GUI 开发、Bug 定位与修复、持续集成配置等方面积累了较为丰富的实践经验。在开发过程中，我能够按照小组约定的分支模型在 `feature/uiqualitydevopsdocs` 分支上进行开发，通过 Pull Request 向 dev 分支合并代码，并按照规范格式填写代码提交说明。在 Issue 驱动的任务管理模式，从任务领取、开发实现到验收关闭形成了完整的闭环。在跨分支协作方面，与组长刘紫媛协作完成了静态资源整理工作，与殷瑞涵协作完成了 WASD 移动路径偏差修复。这些实践经历较好地达到了课程所要求的“培养小组协同开发能力”和“掌握基于 Github 的小组协同开发工具和平台”的目标。

4.4 成员四（谢天）任务分工及实施过程描述

1) 承担任务

在本项目开发过程中，我主要参与后端系统设计与实现、测试体系建设以及代码维护工作，实现了项目从服务端架构搭建到数据持久化实现，再到测试验证与文档完善等多个环节的开发任务，为游戏系统的稳定运行和最终交付提供了技术支撑。

2) 实施过程

开发工作总体按照“服务搭建—业务实现—数据持久化—测试验证—项目优化”的顺序推进。

首先完成 HTTP 服务框架搭建，实现基础请求处理能力和 API 路由功能；随后设计控制器层和会话管理机制，建立前后端通信流程；之后完成数据库结构设计与 JDBC 接入，实现用户和游戏数据持久化；在此基础上开发存档与读档功能，解决玩家状态恢复和关键任务同步问题；最后补充自动化测试、整理回归测试清单、完善代码注释。

3) 完成成果

通过本次开发，完成了项目后端核心功能的实现，建立了较完整的 Web 服务与数据管理体系。

功能方面，负责实现游戏后端服务和控制接口。基于 Java 原生 ServerSocket 搭建轻量级 HTTP 服务器，实现请求处理、静态资源访问和路

由分发功能，并完成用户注册、登录、会话管理、游戏命令调用、状态查询、游戏存档与读档、游戏记录查询等功能，为前端页面提供数据支持。

数据管理方面，负责 MySQL 数据库接入与持久化模块开发。实现用户信息、游戏记录和玩家状态的数据存储功能，并完成数据库自动创建和数据表初始化逻辑，使系统能够在首次运行时自动完成环境配置。

测试方面，补充并完善了 Room、Player、Item、Command 等核心模块测试，整理了登录注册、房间移动、物品拾取、谜题交互、存档读档和完整通关流程等回归测试内容，为项目验收提供了测试依据。

通过本次项目实践，我进一步掌握了 Java Web 开发、数据库持久化、会话管理以及软件测试等相关技术，对完整软件项目的开发流程和团队协作过程有了更加深入的认识。

5. 实施过程问题记录与分析

5.1 前后端状态同步冲突导致物品丢失

问题描述：玩家拾取物品后，房间发生异常跳转或物品凭空消失；读取存档和进入当前进度时，背包中的物品和房间内的物品均不可见。

原因分析：前端 `refreshGameStatus()` 函数在每次与后端交互后调用 `syncFromBackendStatus()`，该函数会用后端返回的数据完整覆盖前端的 `currentRoomId`、`inventory` 和 `roomItems`。然而，后端并不感知前端的虚拟物品（如 `final_key`、`treasure`），也不感知前端对房间物品列表的修改，导致前端状态被后端的“旧数据”覆盖。此外，`startGameFromMenu("load")` 先调用了异步的 `enterGameFromAuth()`，其内部的 `refreshGameStatus` 与后续的 `loadSavedGame()` 存在竞态条件，后到的响应覆盖了已恢复的存档状态。

解决方案：为 `syncFromBackendStatus` 增加 `allowRoomChange` 选项参数。默认为 `false`，此时仅更新辅助数据（如房间描述、出口信息），不覆盖前端的房间 ID、背包和房间物品列表；仅在初始加载、新游戏、读档等场景下设置为 `true`，允许完整同步。同时，`allowRoomChange: true` 时不再覆盖房间物品列表，背包合并改为前端优先策略 `[...gameState.inventory, ...backendInventory]`。对于“load”路径，不再调用 `enterGameFromAuth()`，直接设置变量避免竞态。

经验总结：在前后端分离架构中，当存在前端独有状态时，必须明确“数据权威来源”。本项目最终确立前端为物品位置和背包的权威来源，后端仅作为辅助数据提供者。

5.2 加速移动时房间交界处卡住

问题描述：玩家在持续移动时，到达房间出口附近无法穿越到相邻房间，必须后退一点再慢慢走才能通过。

原因分析：原加速步长为 12 像素，而出口检测阈值 `isAtExit` 仅为 6 像素。当玩家以较大步长移动时，可能直接从出口检测区域内"跳过"，导致 `isAtExit` 始终为 `false`，玩家无法触发房间切换。

解决方案：将加速步长从 12 减小为 9，同时将 `isAtExit` 阈值从 6 增大到 8，缩小步长与阈值的差距。此外，新增 `hasCrossedExit()` 穿越检测函数，通过比较移动前后玩家位置与出口位置的关系，判断玩家是否在移动过程中"穿过"了出口线，即使最终落点不在检测区域内也能正确触发房间切换。

经验总结：在连续移动的游戏系统中，不能仅依赖"落点检测"，必须考虑"路径穿越"的情况，否则高速移动会导致边界检测失效。

5.3 丢弃物品出现在当前房间而非初始房间

问题描述：玩家在大厅丢弃钥匙后，钥匙出现在大厅的拾取列表中，而非钥匙的初始位置校门口。

原因分析：原始的 `drop` 命令实现将物品放回当前房间，这符合一般游戏逻辑，但与项目需求不符。用户明确要求"钥匙丢掉也回校门口"，即物品应回到其初始出现的位置。

解决方案：新增 `ITEM_HOME_ROOM` 映射表，定义每个物品的初始房间。丢弃物品时，前端根据该映射表将物品放回对应房间，而非当前房间。同时，`drop` 命令的前端处理逻辑从 `applyFrontEndCommand` 中独立出来，确保即使后端返回失败，前端也能正确执行丢弃逻辑。

经验总结：游戏物品的位置管理需要根据具体游戏规则设计，不能简单假设"丢弃=放回当前房间"。

5.4 返回上一房间功能失效

问题描述：点击"返回上一房间"按钮没有任何反应。

原因分析：存在两个问题：第一，`applyFrontEndCommand` 函数不是 `async` 函数，其中对 `back` 命令使用的 `await` 无效，导致异步操作未正确执行；第二，`roomHistory`（房间历史栈）仅存在于内存中，未保存到 `localStorage`，页面刷新后历史记录丢失。

解决方案：将 `back` 命令的处理逻辑从 `applyFrontEndCommand` 移至 `handleCommand` 中直接实现，避免异步问题。同时，在 `buildFullState()` 和 `applyFullState()` 中增加对 `roomHistory` 的保存和恢复，确保存档/读档时房间

历史不丢失。

经验总结：在异步编程中，`await` 必须在 `async` 函数内才能生效。对于涉及状态持久化的功能，必须确保所有关键状态都被纳入保存/恢复机制。

5.5 后端传送房间规则的问题

问题描述：前端的传送目标过滤排除了 `locked_room`，但后端 `TransporterRoom.getRandomRoom()` 可能随机到 `LockedRoom`，导致前后端状态不一致。

原因分析：后端的传送房间随机选择逻辑仅排除了自身（`randomRoom == this`），未排除 `LockedRoom`。而前端在传送确认时过滤了上锁房间，导致前端认为传送到了合法房间，但后端记录的目标可能是上锁房间。

解决方案：修改 `TransporterRoom.getRandomRoom()` 的 `do-while` 循环条件，增加 `randomRoom instanceof LockedRoom` 排除条件，使后端与前端的过滤逻辑保持一致。

经验总结：前后端对同一业务逻辑的过滤规则必须保持一致，否则会产生状态分歧。建议将核心业务规则集中定义，前后端共享或通过 API 约定。

5.6 后端物品属性扩展与 API 适配

问题描述：前端需要区分物品的类型（钥匙、食物、工具等）和是否可使用，但后端 `getGameStatus` 接口仅返回物品名称和描述，缺少 `itemType`、`useEffect`、`usable` 等属性。

原因分析：原 `Item` 类设计较为简单，仅包含名称、描述和重量三个基本属性。谜题系统引入后，物品需要支持“使用”操作（如用钥匙开锁、用数据线连接电脑），原有数据结构无法满足需求。

解决方案：在 `Item` 类中新增 `itemType`（物品类型，如 `KEY`、`FOOD`、`TOOL`、`NORMAL`）和 `useEffect`（使用效果描述）两个属性，并提供 `isUsable()` 方法判断物品是否可使用。同时修改 `GameController.getGameStatus()` 方法，在返回的物品信息中增加 `itemType`、`useEffect`、`usable` 三个字段，供前端渲染交互按钮使用。

经验总结：API 设计应考虑前端交互需求，及时扩展数据结构。当业务逻辑复杂度增加时，数据模型需要同步演进，否则前端只能通过硬编码补全信息，增加维护成本。

5.7 自动存档机制不完善

问题描述：玩家拾取物品、使用物品、吃饼干、进入新房间等操作后，

如果未手动保存, "进入当前进度"恢复的状态是上一次自动保存的旧状态, 导致进度丢失。

原因分析: 原实现仅在手动保存和页面关闭 (beforeunload) 时触发保存, 中间的操作未被自动记录。

解决方案: 在 takeItem、dropItem (前端)、useItem (钥匙开宝库)、eatCookie、enterNextRoom、传送确认等关键操作后, 均调用 saveFullState('zuul_auto_' + currentUsername) 进行自动保存。同时, 在 beforeunload 和 logout 事件中也触发自动保存, 确保玩家进度不丢失。

经验总结: 游戏类应用的存档策略应采用"关键操作即保存"的原则, 而非依赖用户手动保存或页面关闭事件, 后者在浏览器崩溃等异常情况下可能无法触发。

6. 任务总结

经过项目组成员的共同开发与测试, 项目已完成预定目标, 实现了从经典 World of Zuul 文本冒险游戏向可视化 Web 游戏系统的升级与扩展。系统整体运行稳定, 各模块之间能够协同工作, 形成了集游戏逻辑、图形界面、用户管理、数据持久化和状态同步于一体的完整校园探索游戏平台。

在游戏功能方面, 项目构建了以校园场景为背景的探索解谜体系, 设计了校园入口、主大厅、图书馆、计算机实验室、森林小路、上锁宝库、宝藏房间以及传送房间等多个功能区域。玩家需要在不同场景之间移动探索, 通过收集钥匙、地图、笔记本、数据线等关键道具, 逐步破解谜题并推进剧情发展, 最终获得最终钥匙完成游戏目标。整个流程将房间探索、物品收集、谜题推理和任务解锁有机结合, 形成了较完整的游戏体验。

在前端交互方面, 项目实现了基于浏览器的可视化游戏界面。系统支持房间背景动态切换、校园地图展示、四方向角色行走动画、方向导航提示、背包管理、物品悬浮显示、状态提示以及谜题弹窗等功能。玩家既可以通过方向按钮进行操作, 也可以使用键盘完成移动控制, 游戏反馈更加直观, 交互体验相比传统命令行模式得到了明显提升。

在后端服务方面, 项目基于 Java 构建轻量级 HTTP 服务, 实现了前后端分离架构。系统提供用户注册、登录、命令执行、状态查询、游戏保存、游戏加载、新游戏创建以及游戏记录查询等多个接口, 能够满足浏览器端与游戏逻辑之间的数据交互需求。同时实现了跨域访问支持、静态资源服务、路径规范化处理以及端口自动切换等功能, 提高了系统的稳定性和可部署性。

在数据管理方面，项目建立了完整的持久化机制。通过 MySQL 数据库对用户信息、游戏记录和玩家状态进行统一管理，实现游戏进度的长期保存与恢复。系统能够记录玩家当前位置、背包物品、访问历史以及关键任务状态，并在加载存档时准确恢复游戏运行环境，使游戏具有持续性和可重复游玩的特点。

在软件工程实践方面，项目采用模块化设计思想，对前端界面、Web 服务、控制器、游戏逻辑、状态管理和数据库访问等功能进行分层组织，降低了模块之间的耦合度，提高了系统的可维护性与扩展性。同时保留并完善了单元测试，对房间管理、玩家行为、命令执行、物品交互等核心功能进行了验证，为项目稳定运行提供了保障。

总体来看，本项目成功实现了经典教学案例向完整 Web 应用的扩展，不仅完成了预期功能目标，而且在用户交互体验、系统架构设计、数据持久化以及工程化实现等方面进行了较为全面的实践。项目综合运用了 Java 程序设计、数据库技术、Web 开发技术和软件工程方法，具备较完整的软件系统形态，也体现了团队在需求分析、系统设计、编码实现和测试验证等方面的综合开发能力。

7. 参考文献

- [1] 郑有庆.Git 在编程教学中的应用探究[J].铜陵职业技术学院学报,2023,22(2):96100.DOI:10.16789/j.cnki.1671752x.2023.02.019.
- [2] 巩瑜.基于 Git 与 GitHub 的代码版本控制规范与协作流程研究[N].重庆科技报,20260127(010).DOI:10.44517/n.cnki.ncqkj.2026.000094.
- [3] 孙成龙,周衍庆,张岩,等.人工智能赋能软件工程教育模式的探索与实践[J].知识窗(教师版),2026,(3):2123.
- [4] 林晓芳.软件工程中 Web 开发技术的实践应用[J].数字技术与应用,2026,44(2):6163.
- [5] 刘耀东,刘翀,马小明.基于大模型的高质量单元测试代码生成框架[J].电子制作,2026,34(1):107111.DOI:10.16589/j.cnki.cn113571/tn.2026.01.018.
- [6] 李波,覃俊,李子茂,等.“人工智能+新工科”视域下软件工程专业实验实践教学改革[J].计算机教育,2021,(7):8286.DOI:10.16512/j.cnki.jsjy.2021.07.020.
- [7] 何晶. Scrum 敏捷方法在软件项目管理中的应用[J].数字技术与应用,2021,39 (3):8789.DOI:10.19695/j.cnki.cn121369.2021.03.29.
- [8] 王红蕾. 基于 DevOps 的轻量级持续交付方案[J].计算机系统应用,2020,29 (9):8794.DOI:10.15888/j.cnki.csa.007540.
- [9] 林斌. 单元测试在软件测试中的应用分析[J].电脑编程技巧与维护,2020, (2):2022.DOI:10.16184/j.cnki.comprg.2020.02.006.
- [10] 张贤坤,苏静,王怡. 新工科背景下软件工程专业实践能力培养体系探索与实践[J].计算机教育,2020, (2):8890.DOI:10.16512/j.cnki.jsjy.2020.02.022.
- [11] 李亚奇.基于软件工程的 Web 开发技术分析[J].软件,2025,46(3):181-183.
- [12] 李玉涛,黄飞龙.基于 Web 自动化测试工具的设计与开发[J].工业控制计算机,2024,37(11):56-58.
- [13] 许文锋.软件工程中 Web 开发技术的应用[J].信息记录材料,2024,25(9):150-152.DOI:10.16009/j.cnki.cn13-1295/tq.2024.09.078.
- [14] 李佳琪.软件工程中的软件开发与编程技术研究[C]//河南省民办教育协会.2024 年高等教育发展论坛论文集(下册).[出版者不详],2024:214-215.DOI:10.26914/c.cnkihy.2024.009262.
- [15] 汪雁,黄伟,安龙,等.Web 前端开发案例实战教程[M].重庆大学出版社:2024.
- [16] 刘悦之.基于 Git 的分布式版本控制系统的设计与实现[J].科技传播,2012,4(22):197-198.
- [17] 李晓蕙.网页游戏中界面设计的研究和应用[D].上海交通大学,2008.
- [18] 和歆雨.基于 HTML5 的网页游戏的设计与开发[J].中国战略新兴产业,2018,(4):94-95.DOI:10.19474/j.cnki.10-1156/f.003193.
- [19] 王真.版本控制工具在软件开发项目管理中的应用——以 GIT 为例[J].项目管理技术,2020,18(6):131-134.
- [20] 徐娅.Git 版本控制工具在团队协作项目中的应用[J].智能计算机与应用,2019,9(5):341-343.

《软件工程实践（二）》成绩评定表

专业、班级	软件 2302 班	小组名称	luckyginger
代码仓库地址	https://github.com/wutkst/kai-fa-luckyginger		
成员一姓名	刘紫媛	学号	1023006395
成员二姓名	殷瑞涵	学号	1023000356
成员三姓名	赵欣然	学号	1023000344
成员四姓名	谢天	学号	1023000175
评价内容		满分	实得分
			得分
实践任务 完成情况	软件架构设计	25	
	软件版本计划与项目管理	25	
	软件分支管理	25	
	代码规范与代码评审	15	
	持续集成与部署	10	
实践任务评分		100	
实践报告 总评情况	学习态度与考勤	10	
	报告格式的规范性	20	
	报告的逻辑结构与语言表达	25	
	实践内容的正确性与合理性	40	
	文献引用及标注	5	
实践报告评分		100	
最终评定成绩（以优、良、中、及格、不及格评定）			

指导教师签字：

2026 年 6 月 20 日